

Unity Ecosystem Integrations

This page describes the integration features with Timeline, Animator, Visual Scripting, Input System, Networking, Scriptable Audio Pipeline, Chunity (Chuck), and Maestro / MPTK.

Namespaces:

Integration	Namespace
Animator	<code>jp.kshoji.unity.midi</code>
Timeline	<code>jp.kshoji.unity.midi.timeline</code>
Visual Scripting	<code>jp.kshoji.unity.midi.visualscripting</code>
Input System	<code>jp.kshoji.unity.midi.integrations.inputsystem</code>
Networking	<code>jp.kshoji.unity.midi.net</code>
Scriptable Audio	<code>jp.kshoji.unity.midi.scriptableaudio</code>
Chunity	<code>jp.kshoji.unity.midi.integrations.chunity</code>
MPTK	<code>jp.kshoji.unity.midi.mptk</code>

Locations:

Integration	Location
Animator	<code>Assets/MIDI/Scripts/Integrations/Animator/</code>
Timeline	<code>Assets/MIDI/Scripts/Integrations/Timeline/</code>
Visual Scripting	<code>Assets/MIDI/Scripts/Integrations/VisualScripting/</code>
Input System	<code>Assets/MIDI/Scripts/Integrations/InputSystem/</code>
Networking	<code>Assets/MIDI/Scripts/Integrations/Networking/</code>
Scriptable Audio	<code>Assets/MIDI/Scripts/Integrations/ScriptableAudio/</code>
Chunity	<code>Assets/MIDI/Scripts/Integrations/Chunity/</code>
MPTK	<code>Assets/MIDI/Scripts/Integrations/MPTK/</code>

Unity-MCP (optional)

Cursor / MCP clients can drive diagnostics and setup via [Unity-MCP](#). Tools live under `Assets/MIDI/Scripts/Integrations/Mcp/` and compile only when `com.ivanmurzak.unity.mcp` is installed (plus NuGet gates). **Not part of Asset Store core distribution.**

Item	Value
Sample prompts	unity-mcp-sample-prompts.md
Checklist	unity-mcp-define-matrix.md
Integration README	../Scripts/Integrations/Mcp/README.md
VST host	Separate package — Unity-VST3-Bridge

Quick path: install Unity-MCP → Play Mode `midi-init` → `midi-devices-list` . For a **built Player**, add `MidiMcpRuntimeBootstrap` , point **Host** at the LAN MCP Server, and use control tools only after Editor wiring (see [MCP README](#)). Networking tools require `FEATURE_MIDI_NETWORK` (`midi-net-*` setup = Editor; `discovery-status` / `rtt` = Player). MIDI 2.0 / MPE / CI tools are always in the core MCP asm (`midi2-*` , `mpe-zone-status` , `midi-ci-discover`). VST host MCP is desktop-oriented in [Unity-VST3-Bridge](#) (not an Android MCP target).

Prerequisites

Timeline Integration

1. Install **Timeline** (`com.unity.timeline`) via the Unity Package Manager
2. Add the scripting define symbol `FEATURE_USE_TIMELINE` in Project Settings

Item	Value
Assembly Definition	<code>jp.kshoji.midi.timeline</code>
Scripting Define Symbol	<code>FEATURE_USE_TIMELINE</code>
Settings Path	Project Settings > Player > Other Settings > Script Compilation > Scripting Define Symbols

When the symbol is not defined, the Timeline integration asmdef and sample code (inside `#if FEATURE_USE_TIMELINE` blocks) are not compiled.

Visual Scripting Integration

1. Install **Visual Scripting** (`com.unity.visualscripting`) via the Unity Package Manager
2. Add the scripting define symbol `FEATURE_USE_VISUALSCRIPTING` in Project Settings

Item	Value
Assembly Definition	<code>jp.kshoji.midi.visualscripting</code>
Scripting Define Symbol	<code>FEATURE_USE_VISUALSCRIPTING</code>
Settings Path	Project Settings > Player > Other Settings > Script Compilation > Scripting Define Symbols

When the symbol is not defined, the Visual Scripting integration asmdef and sample code (inside `#if FEATURE_USE_VISUALSCRIPTING` blocks) are not compiled.

Input System Integration

1. Install **Input System** (`com.unity.inputsystem`) via the Unity Package Manager
2. Add the scripting define symbol `FEATURE_INPUT_SYSTEM` in Project Settings

Item	Value
Assembly Definition	<code>jp.kshoji.midi.inputsystem</code>
Scripting Define Symbol	<code>FEATURE_INPUT_SYSTEM</code>
Settings Path	Project Settings > Player > Other Settings > Script Compilation > Scripting Define Symbols

When the symbol is not defined, the Input System integration asmdef and sample code (inside `#if FEATURE_INPUT_SYSTEM` blocks) are not compiled.

Networking Integration

1. Add `FEATURE_MIDI_NETWORK` (UDP hub/client — no extra package)
2. Optional bridges (packages **not** bundled):
 - Mirror: install Mirror → `FEATURE_MIRROR` (+ `MIRROR`)
 - Netcode: UPM `com.unity.netcode.gameobjects` → `FEATURE_NETCODE` (`MIDI_HAS_NETCODE` via `versionDefines`)
 - WSNet2: copy client + `WSNet2.Runtime.asmdef` → `FEATURE_WSNET2` (`MIDI_HAS_WSNET2` via Editor sync); run Lobby/Game servers separately

Item	Value
Core assembly	<code>jp.kshoji.midi.net</code>
Bridge assemblies	<code>jp.kshoji.midi.net.mirror</code> / <code>.netcode</code> / <code>.wsnet2</code>
Scripting Define Symbol	<code>FEATURE_MIDI_NETWORK</code> (+ optional bridge symbols)
Settings Path	Project Settings > Player > Other Settings > Script Compilation > Scripting Define Symbols

Details: [Networking Integration](#) · [Build Post-processing](#)

Scriptable Audio Pipeline Integration

1. Use **Unity 6000.3 LTS** or later (except WebGL)
2. Add the scripting define symbol `FEATURE_SCRIPTABLE_AUDIO` in Project Settings
3. Optional packages (already bundled in `Packages/manifest.json` when using this repository):
 - `com.unity.burst`
 - `com.unity.collections`

Item	Value
Assembly Definition	<code>jp.kshoji.midi.scriptableaudio</code>
Scripting Define Symbol	<code>FEATURE_SCRIPTABLE_AUDIO</code>
Settings Path	Project Settings > Player > Other Settings > Script Compilation > Scripting Define Symbols

When the symbol is not defined, or on Unity 6.2 and earlier / WebGL, the integration asmdef is not compiled (`ScriptableAudioUtility.IsAvailable == false`). The core `jp.kshoji.midi` can still be built as usual.

For details on the enablement procedure, see [Build Post-processing — Scriptable Audio Pipeline Integration](#).

Chunity (Chuck) Integration

1. Install [Chunity](#) **separately** (this plugin does not bundle the runtime)
2. Copy `Integrations/Chunity/Optional/Chunity.Runtime.asmdef.example` to the Chunity Scripts root as `Chunity.Runtime.asmdef`
3. Add the scripting define symbol `FEATURE_CHUNITY` in Project Settings

Item	Value
Assembly Definition	<code>jp.kshoji.midi.chunity</code>
Scripting Define Symbol	<code>FEATURE_CHUNITY</code>
Settings Path	Project Settings > Player > Other Settings > Script Compilation > Scripting Define Symbols

When the symbol is not defined, the integration asmdef and the sample's `#if FEATURE_CHUNITY` blocks are not compiled. For details on the enablement procedure, see [Build Post-processing — Chunity Integration](#).

Scriptable Audio Generator (additional)

1. Apply the `useBuiltInAudioFilter` patch described in the [Chunity Optional patch guide](#) to Chunity
2. Install Unity package `com.unity.collections` (required by `jp.kshoji.midi.chunity.scriptableaudio`)
3. Add `FEATURE_CHUNITY_SCRIPTABLE_AUDIO` (requires `FEATURE_CHUNITY`)

Item	Value
Assembly Definition	<code>jp.kshoji.midi.chunity.scriptableaudio</code>
Package	<code>com.unity.collections</code>
Scripting Define Symbol	<code>FEATURE_CHUNITY</code> + <code>FEATURE_CHUNITY_SCRIPTABLE_AUDIO</code>
Sample	<code>ChunityGeneratorWorkflowSampleScene</code>

Details: [Chunity Integration](#)

Maestro / MPTK Integration

1. Install Maestro / MidiPlayerTK **separately** (this plugin does not bundle the runtime)
2. Add the scripting define symbol `FEATURE_USE_MPTK` in Project Settings
3. When using **Maestro Pro** APIs, also add `MPTK_PRO`

Item	Value
Assembly Definition	<code>jp.kshoji.midi.mptk</code>
Scripting Define Symbol	<code>FEATURE_USE_MPTK</code> (+ optionally <code>MPTK_PRO</code>)
Location	<code>Assets/MIDI/Scripts/Integrations/MPTK/</code>

`FEATURE_USE_MPTK` enables the Free-tier bridge (virtual MIDI device sink / MPTK-as-input adapters). `MPTK_PRO` further unlocks code gated with `#if MPTK_PRO` that depends on Maestro Pro APIs — for example the `OnMidiEvent` rewrite pipeline (`MptkMidiEventPipeline`), Writer / External player bridges, InnerLoop, ListPlayer, Spatializer, and related Pro samples. Without `MPTK_PRO`, those Pro-only paths stay out of compilation; Free-tier integration still builds when only `FEATURE_USE_MPTK` is set.

For details, see [Maestro / MPTK Integration](#) and [MPTK Integration](#).

Animator Integration

- No additional packages or scripting define symbols are required (included in the core asmdef `jp.kshoji.midi`)

For a detailed enablement procedure, see [Build Post-processing — Unity Integration Packages](#).

Animator Integration

Automatically converts MIDI input into Unity Animator parameters. Performance and system messages are mapped through bindings, while SysEx / Raw UMP are received via the `onMessage` callback.

Components

Component	Role
<code>MidiAnimatorDriver</code>	MIDI reception → Animator parameters + <code>onMessage</code>
<code>MidiAnimatorMapping</code>	ScriptableObject. Binding definitions can be shared
<code>MidiBlendTreeDriver</code>	Feeds 2–4 channel CC into Blend Tree axes

Two-layer model

Layer	Target	Use
Bindings	MIDI mappable to Float / Int / Bool / Trigger	CC, notes, Program Change, Clock, etc.
<code>onMessage</code>	All messages (including SysEx / Raw UMP)	Custom scripts, payload processing

`onMessage` and bindings can be used together for the same message. To avoid double processing, use only one of them.

Binding (`MidiParameterBinding`)

Field	Description
<code>messageType</code>	<code>MidiOutgoingMessageType</code> (NoteOn / ControlChange / TimingClock, etc.)
<code>group</code>	0–15, <code>-1</code> = all groups
<code>channel</code>	0–15, <code>-1</code> = all channels (ignored for group-only messages such as Clock / Start)
<code>controllerOrNote</code>	Note / CC / Program number, etc.
<code>valueFilter</code>	Value filter, <code>-1</code> = all
<code>mapMessageValue</code>	Maps NoteOn velocity to Float/Int (equivalent to the old Velocity)
<code>animatorParameterName</code>	Target Animator parameter name
<code>parameterType</code>	Float / Int / Bool / Trigger
<code>responseCurve</code>	Normalized input 0–1 → output
<code>smoothingTime</code>	Smoothing seconds for CC / Pitch Bend, etc.
<code>deviceIdFilter</code>	Device ID filter (empty = all)

Mapping rules

messageType	Recommended parameterType	Behavior
NoteOn / NoteOff	Bool / Trigger	On/Off or Trigger (only on On)
NoteOn + <code>mapMessageValue</code>	Float / Int	Velocity normalization
ControlChange / PitchWheel / Aftertouch	Float	0–1 normalization + curve
ProgramChange / SongSelect, etc.	Int	Sets <code>number</code> directly
TimingClock / Start / Stop / Reset, etc.	Trigger	<code>SetTrigger</code> on reception
SystemExclusive / Raw UMP	—	<code>onMessage</code> only

Note: Binding TimingClock to a Trigger fires on every tick. Be mindful of the timing at which the Animator Trigger is consumed, or thin out the ticks in `onMessage`.

Setup

1. Attach `MidiAnimatorDriver` to a GameObject that has an Animator
2. Create a mapping via `Assets > Create > MIDI > Animator Mapping` (optional)
3. Configure the bindings (Message Type / Group / Channel in the Inspector)
4. If SysEx / UMP is needed, connect a listener to `onMessage`
5. Call `MidiManager.InitializeMidi()` in the scene bootstrap

Binding examples

Use	messageType	controllerOrNote	parameterType	Parameter name
Motion speed	ControlChange	1	Float	Speed
Jump	NoteOn	60	Trigger	Jump
Strike strength	NoteOn + <code>mapMessageValue</code>	60	Float	StrikePower
Tone number	ProgramChange	5	Int	Program
Playback start	Start	—	Trigger	TransportStart

`onMessage` example

```
driver.onMessage.AddListener(args =>
{
    if (args.message.messageType == MidiOutgoingMessageType.SystemExclusive)
    {
        var payload = args.message.payload;
        // SysEx processing
    }
});
```

MidiBlendTreeDriver

For XY pads and DJ controllers. Specify 2–4 CCs and feed 0–1 normalized values into the corresponding Animator float parameters (default: `BlendX` / `BlendY`).

Timeline Integration

Play SMF, record MIDI, and generate markers on the Unity Timeline.

Tracks and clips

Type	Class	Binding	Description
Playback track	<code>MidiPlaybackTrack</code>	<code>SmfPlayer</code>	Plays SMF clips synchronized to Timeline time
Playback clip	<code>MidiPlaybackClip</code>	—	<code>MidiSequenceAsset</code> reference, BPM, channel remap, <code>mute</code> / <code>solo</code>
Record track	<code>MidiRecordTrack</code>	<code>MidiRecorder</code>	Records MIDI input during the clip interval
Record clip	<code>MidiRecordClip</code>	—	Start/Stop recording while the clip is active

Setup (playback)

1. Place a `PlayableDirector` and `SmfPlayer` on a `GameObject`
2. Select **Add > MIDI Playback Track** in the Timeline window
3. Assign `SmfPlayer` to the track's Binding
4. Place a **Midi Playback Clip** on the track
5. Set a `MidiSequenceAsset` in the clip's `Sequence Asset`
6. Confirm in Play mode that the Timeline and SMF play in sync

Time synchronization

The clip-local time of the Timeline is the master for `SmfPlayer.Seek()`.

- During Play: when the difference exceeds 30 ms, it follows by seeking
- Pause / scrub: pauses the `SmfPlayer` and seeks to the position

Per-clip output control is possible via `MidiPlaybackClip`'s `outputChannel` (`-1` = keep original channel), `mute`, and `solo`.

Limitations: Piano-roll editing and direct editing of UMP clips on the Timeline are not supported.

Markers

Marker	Description
<code>MidiMarker</code>	Sends a MIDI message at the specified position (<code>MidiTimelineNotificationReceiver</code> required)
<code>MidiSignalEmitter</code>	Sends a MIDI message together with a Timeline Signal
<code>MidiBarMarker</code>	Bar head. Displays time signature information as a label
<code>MidiTempoMarker</code>	Tempo change position

`MidiMarker` / `MidiSignalEmitter` set what to send via `MidiTimelineOutgoingMessage`. In addition to MIDI 1.0 performance and system messages, they also support SysEx / System Common / Raw UMP (MIDI 2.0).

Setup (send marker)

1. Add `MidiTimelineNotificationReceiver` to the same `GameObject` as the `PlayableDirector` (also possible via `Window > MIDI > Timeline > Add Notification Receiver To Director`)
2. Place a `MidiMarker` or `MidiSignalEmitter` on the Timeline's Marker Track
3. Set the Message Type / Group / Channel / Payload (for SysEx / UMP) in the Inspector

Auto-generation: Select a `MidiSequenceAsset` in the Project window and run `Window > MIDI > Timeline > Generate Markers From Sequence Asset`. Use it with the `PlayableDirector` open in the Timeline window.

Signal integration

Set `MidiTimelineSignalHandler` as the target of a Signal Receiver and call the following from a Timeline Signal:

- `SendNoteOn(int note, int velocity)`
- `SendNoteOff(int note, int velocity)`
- `SendControlChange(int controller, int value)`
- `SendProgramChange(int program)`
- `SendSystemExclusive(byte[] payload)`
- `SendRawUmp(uint[] umps)`
- `SendOutgoingMessage(MidiTimelineOutgoingMessage message)` — supports all message types

Visual Scripting Integration

Build MIDI input/output on a Script Graph without writing code.

Bridge

Add `MidiVisualScriptingBridge` to a GameObject that has a Script Graph attached. It forwards events from `MidiManager` to the Visual Scripting Event Bus. Typed event nodes and the generic `On MIDI Message` can be used together, but connecting both to the same message causes double execution, so use only one of them.

Event nodes (Events > MIDI)

Node	Output
On MIDI Message	deviceId, messageType, group, channel, number, value, data3, payload, payloadLength — all MIDI 1.0 / SysEx / Raw UMP
On MIDI Note On	group, channel, note, velocity, deviceId
On MIDI Note Off	group, channel, note, velocity, deviceId
On MIDI Control Change	group, channel, controller, value, deviceId
On MIDI Program Change	group, channel, program, deviceId
On MIDI Pitch Bend	group, channel, amount (0–16383), deviceId

Action nodes

Category	Node
MIDI > Send	Send MIDI Message (generic DTO + payload), Send MIDI Note On / Off / Control Change / Program Change / Pitch Bend (all support <code>group</code>)
MIDI > Playback	SMF Player Play / Stop
MIDI > Utility	Get Active MIDI Notes, Serialize MIDI UMP Words , Deserialize MIDI UMP Words
MIDI > Note	Note Name To Number / Number To Note Name / Is Note In Scale

Send MIDI Message payload examples

messageType	payload
SystemExclusive	SysEx byte sequence (<code>F0 ... F7</code>)
SystemCommonMessage	System Common byte sequence
Midi2RawUmp	A byte sequence concatenating UMP words in big-endian 4-byte units. Can also be generated with the <code>Serialize MIDI UMP Words</code> unit

Setup

1. Install Visual Scripting via the Package Manager

2. Attach a `Script Machine` and `MidiVisualScriptingBridge` to a `GameObject`
3. Create a Script Graph and place event nodes from **Events > MIDI**
4. Call `MidiManager.InitializeMidi()` in the scene bootstrap
5. Confirm in Play mode that MIDI input → the graph executes

Is Note In Scale

Uses `MidiScaleUtility`. Supports `Major` / `NaturalMinor` / `MajorPentatonic` / `MinorPentatonic`.

Input System Integration

Using the Unity Input System's Synthetic Device and `.inputactions` , connect MIDI input/output to your existing Input Action-based game logic.

Components

Component	Direction	Role
<code>MidiInputSystemBridge</code>	MIDI → Input System	Injects received MIDI into the Synthetic Device. SysEx / Raw UMP via the <code>onMessage</code> callback
<code>InputSystemToMidiBridge</code>	Input System → MIDI	Sends Input Actions as <code>MidiOutgoingMessage</code>
<code>MidiSyntheticDevice</code>	—	128 notes + 128 CC + channel-dedicated axes
<code>MidiInputSystemMapping</code>	—	A ScriptableObject of Action names and MIDI conditions

Synthetic Device layout

Control	Path example	Content
<code>note[128]</code>	<code><MidiSynthetic>/note60</code>	Note 0–1 (velocity / Poly AT)
<code>cc[128]</code>	<code><MidiSynthetic>/cc1</code>	CC 0–1
<code>pitch[16]</code>	<code><MidiSynthetic>/pitch0</code>	Pitch Bend 0–1 (0.5 = center)
<code>program[16]</code>	<code><MidiSynthetic>/program0</code>	Program Change 0–1
<code>channelPressure[16]</code>	<code><MidiSynthetic>/channelPressure0</code>	Channel Aftertouch 0–1
<code>systemPulse[16]</code>	<code><MidiSynthetic>/systemPulse0</code>	Pulses for Clock / Start / Stop, etc. (group index)

SysEx / System Common / Raw UMP cannot be mapped to the button/axis model, so they are not supported by the Synthetic Device. Receive them as a `MidiOutgoingMessage` DTO via `MidiInputSystemBridge.onMessage` .

Mapping (MIDI → Input System)

Create a `MidiInputSystemMapping` via `Assets > Create > MIDI > Input System > Mapping` .

Field	Description
<code>actionName</code>	Action name within <code>.inputactions</code> (for documentation)
<code>messageType</code>	<code>MidiOutgoingMessageType</code> (NoteOn / ControlChange / PitchWheel, etc.)
<code>group</code>	0–15, <code>-1</code> = all groups
<code>channel</code>	0–15, <code>-1</code> = all channels
<code>controllerOrNote</code>	Note / CC number / match condition
<code>valueFilter</code>	Value filter, <code>-1</code> = all
<code>targetControlIndex</code>	CC fallback target (<code>-1</code> = <code>controllerOrNote</code>)
<code>preferDedicatedControl</code>	If <code>true</code> , use pitch / program / channelPressure / systemPulse
<code>invertAxis</code>	Inverts the normalized axis

Reverse binding (Input System → MIDI)

The `InputToMidiBinding` of `InputSystemToMidiBridge` uses a common DTO format.

Field	Description
<code>messageType</code> / <code>canceledMessageType</code>	Send type on performed / canceled
<code>group</code> / <code>channel</code> / <code>number</code> / <code>value</code> / <code>data3</code>	<code>MidiOutgoingMessage</code> fields
<code>useActionValue</code>	Converts the Action value to 0–127 (0–16383 for Pitch)
<code>sendOnPerformed</code> / <code>sendOnCanceled</code>	Send timing

The old fields `note` / `controller` / `sendControlChange` are still read.

Setup

1. Install Input System via the Package Manager
2. Define `FEATURE_INPUT_SYSTEM`
3. Add `MidiInputSystemBridge` to a GameObject and assign a `MidiInputSystemMapping` if needed
4. Specify `<MidiSynthetic>/note60`, etc. in the Binding Path of `.inputactions`
5. If output is needed, add `InputSystemToMidiBridge`
6. If SysEx / UMP is needed, connect a listener to `onMessage`

Sample

Scene	Path
Input System bridge	<code>Assets/MIDI/Samples/Integrations/InputSystem/Scenes/InputSystemBridgeSampleScene.unity</code>

Scriptable Audio Pipeline Integration

Provides the Unity 6.3+ [Scriptable Audio Pipeline](#) as an **optional foundation** for realtime audio generation coordinated with MIDI timing. It includes a metronome Generator, a DSP clock bridge, and Pipe communication DTOs as reference implementations.

Quick start

- 1. Following the [enablement procedure](#), define `FEATURE_SCRIPTABLE_AUDIO`
- 2. Add **Scriptable Audio Bootstrap** (`ScriptableAudioBootstrap`) to an empty GameObject
- 3. Play mode — confirm that you hear a 120 BPM click sound

Or open the sample scene:

`Assets/MIDI/Samples/Integrations/ScriptableAudio/Scenes/ScriptableAudioMetronomeSampleScene.unity`

Components / API

Type	Role
<code>ScriptableAudioBootstrap</code>	Wires up AudioSource + Generator + Bridge all at once
<code>ScriptableAudioUtility</code>	<code>IsAvailable</code> , <code>EnsureAudioSource</code> , <code>AttachGenerator</code> , <code>ValidateSetup</code>
<code>MidiMetronomeGenerator</code>	Reference <code>IAudioGenerator</code> (metronome click)
<code>MidiDspClockBridge</code>	Sends Transport to the Generator via a DSP snapshot + Pipe
<code>MidiDspClockSnapshot</code>	A read-only timing DTO for game logic
<code>MidiDspSequenceScheduler</code>	SMF tick → DSP sample scheduling + Pipe note events
<code>MidiDspSequenceClockMode</code>	<code>SmfTempoMap</code> / <code>ExternalClock</code> (exclusive switching of the tempo axis)
<code>MidiDspSequenceMidiOutBridge</code>	Mirrors scheduled notes to <code>MidiManager</code> (main-thread dispatch)
<code>MidiDspUmpSequenceScheduler</code>	UMP clip tick → DSP sample scheduling + Pipe
<code>MidiDspUmpSequenceClockMode</code>	<code>UmpTempoMap</code> / <code>ExternalClock</code>
<code>MidiDspUmpMidi2OutBridge</code>	Mirrors scheduled UMP packets to <code>MidiManager.SendMidi2RawUmp</code> (frame granularity)
<code>MidiDspUmpSequenceBootstrap</code>	Wires up AudioSource + UMP Scheduler + reference synth + Clock all at once
<code>UmpSequenceSynthGenerator</code>	Reference <code>IAudioGenerator</code> (UMP polyphonic, for verification)
<code>UmpSequenceAsset</code>	A ScriptableObject holding <code>.midi2</code> binary (<code>ToUmpSequence()</code>)
<code>MidiDspUmpSequenceSmfFallback</code>	A fallback that delegates to the SMF Scheduler via <code>SequenceConverter</code>
<code>MptkDspUmpSequenceOutput</code>	UMP Out → MPTK virtual device (<code>FEATURE_USE_MPTK</code> ; under <code>MPTK/ScriptableAudio/</code>)
<code>ScriptableAudioSetupReport</code>	Setup diagnostics (Error / Warning)

Sample demo

Mode	Content
Standalone	Clicks at <code>MidiDspClockBridge.fallbackBpm</code> (default 120)
External Clock	Switch via the sample UI. Inject Start / Timing Clock / Stop from the virtual device <code>virtual:scriptable-audio-sample</code>

SMF sequence playback (DSP sync)

`MidiDspSequenceScheduler` maps the SMF Note On/Off to absolute sample positions on the Unity DSP clock and plays them with the reference synth `MidiSequenceSynthGenerator`. It is a separate layer from `SmfPlayer` (frame-driven, MIDI device output).

Mode	Content
SMF tempo map	Local Transport (Play/Stop/Seek). Tempo comes from meta events in the SMF + <code>tempoFactor</code>
External Clock	<code>clockMode = ExternalClock + MidiClockSync</code> . Follows Start/Stop; BPM is scaled by <code>EstimatedBpm / externalClockReferenceBpm</code>
Hardware MIDI output	Optionally add <code>MidiDspSequenceMidiOutBridge</code> . Mirrors the same schedule as the Pipe to <code>MidiManager</code> (frame granularity)

Sample: `Assets/MIDI/Samples/Integrations/ScriptableAudio/Scenes/ScriptableAudioSequenceSampleScene.unity`

Both coexist with `MidiManager.InitializeMidi()`.

UMP sequence playback (DSP sync)

`MidiDspUmpSequenceScheduler` maps the Note On/Off and UMP channel voices of a `UmpSequence` (`.midi2` clip) to absolute sample positions on the Unity DSP clock and plays them with the reference synth `UmpSequenceSynthGenerator`. It is a **separate layer** from `UmpSequencer` (wall-clock + dedicated-thread driven).

Mode	Content
UMP tempo map	Local Transport (Play / Pause / Stop / Seek). Tempo comes from Flex <code>Set Tempo</code> + PPQ + <code>tempoFactor</code>
External Clock	<code>clockMode = ExternalClock + MidiClockSync</code> . Follows Start/Stop; BPM is scaled by <code>EstimatedBpm / externalClockReferenceBpm</code>
Hardware MIDI 2.0 output	Optionally add <code>MidiDspUmpMidi2OutBridge</code> . Mirrors the same schedule as the Pipe as raw UMP packets to <code>MidiManager.SendMidi2RawUmp</code> (frame granularity)
System / SysEx	With <code>scheduleSystemMessages = true</code> , schedules UMP System (type <code>0x1</code>). Data SysEx (type <code>0x3</code>) is supported both as completed and split/reassembled forms via Pipe (inline ≤52B) and <code>UmpOut</code> mirroring

When to use which:

Use	Recommended
Clip editing / recording / .midi2 trial playback	UmpSequencer
In-game BGM / loops / Seek / DSP-synced audio	MidiDspUmpSequenceScheduler
MIDI 1.0 SMF only	MidiDspSequenceScheduler
Hardware MIDI 2.0 output (frame granularity acceptable)	MidiDspUmpMidi2OutBridge + MidiManager.SendMidi2RawUmp

Sample: Assets/MIDI/Samples/Integrations/ScriptableAudio/Scenes/ScriptableAudioUmpSequenceSampleScene.unity

Coexists with MidiManager.InitializeMidi() / InitializeMidi2(). When using the UMP output bridge, call InitializeMidi2() and a MIDI 2.0 output device must be connected.

Quick start (UMP)

- 1. Following the [enablement procedure](#), define FEATURE_SCRIPTABLE_AUDIO
- 2. Add **Midi Dsp Ump Sequence Bootstrap** (MidiDspUmpSequenceBootstrap) to an empty GameObject
- 3. Assign a UmpSequence in code, or use the demo factory of the sample scene
- 4. Play mode — confirm that notes are audible through the reference synth

```
var bootstrap = gameObject.AddComponent<MidiDspUmpSequenceBootstrap>();
bootstrap.sequence = UmpSequenceReader.ReadSequence(stream)[0];
bootstrap.EnsureReady();
bootstrap.Scheduler.Play();
```

To turn .midi2 into an asset, create a UmpSequenceAsset (menu **Assets > Create > MIDI > Import UMP Sequence Asset From File**, or CreateAssetMenu) and pass the result of asset.ToUmpSequence() to the Bootstrap / Scheduler. For details, see [SMF Tools — UmpSequenceAsset](#).

Items you can check in the sample UI

UI	Content
C major scale (MIDI 1.0 / MIDI 2.0)	Note On/Off playback (MIDI 2.0 uses 32-bit velocity)
Tempo change (120 → 90 BPM)	No beat drift across the Flex Set Tempo section
Loop first 4 notes	loopStartTick / loopEndTick (0–1920)
Tempo factor	Playback speed multiplier
External MIDI Clock	Follows Start/Stop
Mirror UMP packets	Enables MidiDspUmpMidi2OutBridge (the bridge is created automatically after toggling)
Validate Setup	Diagnostic log equivalent to ScriptableAudioUtility.ValidateUmpSequenceSetup

Extensions

Feature	Usage
Group Mute / Solo	<code>MidiDspUmpSequenceScheduler.SetGroupMute(group, mute)</code> / <code>SetGroupSolo(group, solo)</code> (Group 0–15)
MIDI 2.0 controls / attributes	<code>scheduleMidi2Controls</code> / <code>scheduleNativeUmpPackets</code> (sends CC / PC / PNC, etc. via Pipe or raw UMP)
SMF fallback	Bootstrap's <code>playbackMode = SmfDelegated . SequenceConverter.ConvertUmpSequence</code> → <code>MidiDspSequenceScheduler</code>
MPTK integration	<code>MptkDspUmpSequenceOutput</code> in <code>MPTK/ScriptableAudio/ (FEATURE_USE_MPTK + FEATURE_SCRIPTABLE_AUDIO)</code> . Routes UMP Out to an MPTK virtual device

Known limitations (reference synth / Pipe)

Item	Behavior
System (type <code>0x1</code>)	Can be dispatched to Scheduler → Pipe / UmpOut. Not processed by the reference synth (no sound)
Data 128-bit (type <code>0x5</code>) and 3-word or more	<code>umpPacketTable</code> + full UmpOut mirror . Pipe inline and the reference synth support 1–2 words only
UMP Stream (type <code>0xF</code>)	Out of scope
Hardware output precision	Frame granularity (different from the DSP sample precision of the built-in synth)
WebGL / Unity 6.2 and earlier	Integration not supported (same as "Limitations" below)

Manual verification checklist (sample scene)

Open `ScriptableAudioUmpSequenceSampleScene` in Play Mode and confirm the following.

- 1. The MIDI 1.0 / MIDI 2.0 scales sound through the reference synth
- 2. In the tempo-change demo, beats do not drift and accumulate
- 3. With loop ON, the 4-note section repeats / it can resume after Seek
- 4. With External Clock ON, it follows the virtual device's Start/Stop
- 5. Validate Setup has no Errors, and no bridge-missing Warning appears when UMP Out is OFF
- 6. (Optional) With UMP Out ON + `InitializeMidi2()` + a real device, raw UMP arrives
- 7. (Optional) Play a real `.midi2` via `UmpSequenceAsset` and confirm it sounds roughly the same as `UmpSequencer`

The Edit Mode self-checks (Pipe blittable, tick ↔ DSP, Extractor, risk mitigation) are included in **Window > MIDI > Validate Scriptable Audio Setup** or `ScriptableAudioUtility.ValidateFoundation()`. Automated regression via the Unity Test Runner has not been added.

For details on the implementation layout and diagnostic API, also see [Scriptable Audio Integration](#).

Diagnostics

- Code: `ScriptableAudioUtility.ValidateSetup(gameObject)` → `ScriptableAudioSetupReport.ToSummary()`
- UMP only: `ScriptableAudioUtility.ValidateUmpSequenceSetup(gameObject)`
- Editor: **Window > MIDI > Validate Scriptable Audio Setup** (validates the selected GameObject, or the first Bootstrap in the scene when nothing is selected. Automatically distinguishes metronome / SMF / UMP)

Limitations

- **WebGL:** The integration asmdef is excluded. It does not cause a build error; the API is `IsAvailable == false`
- **Unity 6.2 and earlier:** Not compiled due to source guards
- **Double driving the same output:** Do not use `UmpSequencer` and `MidiDspUmpSequenceScheduler` simultaneously for the same music output (unless intentional)
- Details: [Platforms — Scriptable Audio](#) / [Scriptable Audio Integration](#)

Chunity (Chuck) Integration

An **optional integration** that connects ChuckK patches on an externally installed [Chunity](#) to this plugin's MIDI I/O, SMF, Clock, and Timeline / Visual Scripting. Chuck's built-in `MidiIn` depends on OS device numbers and does not pass through this plugin's virtual devices, filters, or network routes, so the core of the integration is **Unity MIDI** → **Chuck global variables / Events**.

Official API reference: [Chunity Documentation](#)

Quick start

1. Following the [enablement procedure](#), prepare Chunity and `FEATURE_CHUNITY`
2. Place a `ChuckMainInstance` (and optionally `ChuckSubInstance`)
3. Assign a `.ck` / inline patch to `MidiChuckPatchHost` on the same GameObject (or referenced target)
4. Add `MidiChuckBridge` — by default **Use Default Convention** is on
5. Send MIDI (a real device or virtual Inject) in Play mode and confirm the patch responds

Components

Component	Role	Phase
<code>MidiChuckInstanceTarget</code>	Main / Sub dispatch (String / array / associative array / RunFile args / ListenOnce, etc.)	A
<code>MidiChuckBridge</code>	MIDI → SetInt / SetFloat / Array / Associative + SignalEvent	1, A
<code>MidiChuckPatchHost</code>	Runs inline / TextAsset / StreamingAssets <code>.ck</code> (Context Menu "Run Patch Now")	1
<code>MidiChuckMapping</code>	MIDI condition → Chuck global / Event (including <code>AssociativeInt</code> / <code>AssociativeFloat</code>)	1, A
<code>MidiChuckUtility</code>	Default global names and <code>IsAvailable</code> / <code>SetupHint</code> / <code>SetChuckLogLevel</code>	1, E
<code>MidiChuckEventToMidi</code>	Chuck Event → UnityEvent / MIDI OUT (scalar Get + array read <code>arraySource</code>)	2, D
<code>MidiChuckSmfLink</code>	<code>SmfPlayer</code> → virtual device (default <code>virtual:chuck-smf</code>) → Bridge	2
<code>MidiChuckClockSync</code>	<code>MidiClockSync</code> → <code>bpm</code> / <code>beat</code> / <code>bar</code> / transport Event	2
<code>MidiChuckPolyVoiceHelper</code>	Polyphonic voiceld (+ optionally <code>activeNotes[]</code> , etc.)	2
<code>MidiChuckFloatSyncer</code> / <code>MidiChuckIntSyncer</code> / <code>MidiChuckStringSyncer</code>	<code>Chuck*Syncer</code> wrappers (specify global names from the Inspector)	B
<code>MidiChuckFloatArraySyncer</code> / <code>MidiChuckIntArraySyncer</code>	Array Syncer wrappers	B
<code>MidiChuckParameterPoller</code>	Read pair for ParameterBinder (multi-parameter polling)	B
<code>MidiChuckHostAdvancer</code>	Unity master timeStep / pos / tick Event bridge	C
<code>MidiChuckPatchLifecycle</code>	Cooperative stop Event + shred cleanup + optional Restart	C
<code>MidiChuckSampleBank</code>	Program / Note / Bank → sample path → SetString + play Event	D
<code>MidiChuckPreset</code>	float/int snapshot (+ optional patch)	3
<code>MidiChuckParameterBinder</code>	Preset application, live SetFloat / SetInt	3
<code>MidiChuckMarker</code> , etc.	Timeline markers / parameter clips (including Phase E actions)	3, E (<code>FEATURE_USE_TIMELINE</code>)

Chuck VS Units	Set/Get String / array, RunFile, Event listen, HostAdvancer, PatchLifecycle, SampleBank, etc.	3, E (<code>FEATURE_USE_VISUALSCRIPTING</code>)
----------------	---	--

Default convention (when no mapping matches)

MIDI	ChuckK
Note On	<code>midiNote</code> , <code>midiVelocity</code> , Event <code>noteOn</code>
Note Off	<code>midiNote</code> , Event <code>noteOff</code>
CC	<code>ccNumber</code> , <code>ccValue</code> , Event <code>controlChange</code> (optionally <code>cc[]</code>)
Pitch Bend	<code>pitchBend</code> (-1..1)
Program Change	<code>program</code>

A minimal contract example on the sample `.ck` side:

```
global Event noteOn;
global Event noteOff;
global int midiNote;
global float midiVelocity;

SinOsc osc => dac;
while (true) {
    noteOn => now;
    Std.mtof(midiNote) => osc.freq;
    midiVelocity => osc.gain;
}
```

Mapping (`MidiChuckMapping`)

Use `MidiChuckBinding` to declare "which MIDI condition corresponds to which Chuck variable / Event." Combine `messageType` / `group` / `channel` / `controllerOrNote` (all `-1` = all) with the write target (int / float / Event / float array), `floatScale` , and `broadcastEvent` . You can swap in a different `.ck` without code changes.

Choosing between Phase 2–3

Use	Recommended
SMF → Chuck	Align the virtual device with <code>MidiChuckSmfLink</code> and match <code>SmfPlayer.outputDeviceId</code> to the Bridge's <code>deviceIdFilter</code>
External Clock	<code>MidiChuckClockSync</code> (BPM → <code>bpm</code> , beat / bar / Start-Stop Event)
Polyphony	<code>MidiChuckPolyVoiceHelper</code> + Bridge's <code>useDefaultConvention = false</code> (prevents double firing with the monophonic convention)
Preset UI	<code>MidiChuckPreset</code> + <code>MidiChuckParameterBinder</code> (sample: <code>ChunityPresetsSampleScene</code>)
Chuck → MIDI OUT	<code>MidiChuckEventToMidi</code>

Timeline / Visual Scripting (Phase 3 / E)

Additional symbol	Assembly	Content
FEATURE_USE_TIMELINE	jp.kshoji.midi.chunity.timeline	MidiChuckMarker (SetString / SetArray / RunFile / HostAdvancer / Lifecycle* / SampleBank, etc.) / MidiChuckTimelineNotificationReceiver / MidiChuckParamTrack
FEATURE_USE_VISUALSCRIPTING	jp.kshoji.midi.chunity.visualscripting	Units in the MIDI / Chunity category (Phase E: Get* / RunFile / Event listen / HostAdvancer / PatchLifecycle / SampleBank, etc.). Register: Window → MIDI → Visual Scripting → Register Chunity Nodes

Editor diagnostics (no additional define required): **Window** → **MIDI** → **Chunity** → **Diagnostics** — log level and integration state.

Scriptable Audio Generator (additional option)

Leaving the control path (Bridge / Mapping / Preset, etc.) as is, this replaces only the audio output from OnAudioFilterRead with the Unity 6.3+ **Scriptable Generator** (IAudioGenerator). It is a separate symbol from the main Scriptable Audio integration such as the metronome (FEATURE_SCRIPTABLE_AUDIO).

Type	Role
ChuckMainGeneratorDriver	Drives the Main VM with AudioSource.generator
ChuckSubGeneratorDriver	Drives the Sub (with spatialization) with the Generator
ChuckAudioOutputMode	Auto / FilterRead / ScriptableGenerator

Mode	Behavior
Auto (recommended)	Generator in supported environments, otherwise (WebGL, etc.) the traditional FilterRead
FilterRead	The same OnAudioFilterRead as standard Chunity
ScriptableGenerator	Strictly uses the Generator (falls back with a one-time warning when unsupported)

Setup:

1. Apply the patch from the [enablement procedure \(Scriptable Audio Generator\)](#), install com.unity.collections , and add FEATURE_CHUNITY_SCRIPTABLE_AUDIO
2. Add ChuckMainGeneratorDriver to ChuckMainInstance (add ChuckSubGeneratorDriver to each ChuckSubInstance when using Sub)
3. Set the **Audio Output Mode** (usually Auto)
4. Play — the Driver sets useBuiltInAudioFilter = false while the Generator is connected to prevent double playback

Operational notes:

Item	Content
No double playback	Disable FilterRead when the Generator is enabled. Do not drive them simultaneously
Main → Sub order	When both are Generator , <code>ChuckGeneratorTracker</code> allows the Sub after the Main VM has advanced. When the Main is FilterRead, it depends on the Chunity Mixer order
Microphone (adc)	The Main Driver supplies the recording buffer to the DSP from the main thread via a lock-free ring
Mute	Respects <code>AudioSource.mute</code> . Advances the VM clock while muting output only
Sample rate	Warns when the ChuckK initialization rate differs from the Generator <code>AudioFormat.sampleRate</code>

Limitations / roadmap

- The Chunity runtime is **not bundled**. All three are required: installation, `Chunity.Runtime.asmdef`, and `FEATURE_CHUNITY`
- If only the symbol is enabled but Chunity is not installed, type resolution errors occur (same as MPTK)
- Users can always call the raw Chunity API directly. This plugin is a thin bridge layer in the MIDI domain
- **Not implemented (roadmap)**: MIDI 2.0 / UMP mapping, InstanceTarget's String / Once / GetArray / RunFile arguments, Syncer / associative array / `*_AT` writes, VM / shred control helpers, UGen probe, Host Time Advancer, etc. — see [Future features in the table of contents](#)
- Detailed procedure: [Chunity Integration](#) / [Build Post-processing](#)

Networking Integration

LAN MIDI event and SMF playback sync. Core path is UDP (`FEATURE_MIDI_NETWORK`). Optional bridges reuse `MidiNetworkMessage` / `MidiNetworkMessageCodec` on Mirror, Netcode for GameObjects, or [WSNet2](#).

Core (UDP)

Component	Role
<code>MidiNetworkHub</code>	Host; <code>OnOutboundMessage</code> feed for bridges
<code>MidiNetworkClient</code>	Inject received events into a virtual device
<code>MidiPlaybackSync</code>	<code>SmfPlayer</code> position sync
<code>MidiNetworkMessageCodec</code>	Shared binary codec for optional bridges

Optional bridges

Component	Transport	Authority
<code>MidiMirrorBridge</code>	Mirror <code>NetworkBehaviour</code>	Host / server
<code>MidiNetcodeBridge</code>	NGO <code>NetworkBehaviour</code>	Host / server
<code>MidiWSnet2Bridge</code>	WSNet2 <code>Room</code> RPC	Master

Install the framework locally; do not commit Mirror / NGO / WSNet2 into distribution branches. Shared wiring: assign Master/Host `hub` , peer `playbackSync` , then exercise sample scenes. Bridges subscribe to hub **only while Host/Master**. `ForwardTo*` returns `MidiNetworkTransmitResult` .

Full steps: [Networking Integration](#)

Sample scenes

Scene	Path
Animator integration	Assets/MIDI/Samples/Integrations/Scenes/MidiAnimatorIntegrationSampleScene.unity
Timeline integration	Assets/MIDI/Samples/Integrations/Scenes/MidiTimelineIntegrationSampleScene.unity
Visual Scripting integration	Assets/MIDI/Samples/Integrations/Scenes/MidiVisualScriptingIntegrationSampleScene.unity
Input System bridge	Assets/MIDI/Samples/Integrations/InputSystem/Scenes/InputSystemBridgeSampleScene.unity
Network MIDI (UDP)	Assets/MIDI/Samples/Integrations/Networking/Scenes/MidiNetworkJamSampleScene.unity
Network MIDI (Mirror)	Assets/MIDI/Samples/Integrations/Networking/Scenes/MidiMirrorNetworkSampleScene.unity
Network MIDI (Netcode)	Assets/MIDI/Samples/Integrations/Networking/Scenes/MidiNetcodeNetworkSampleScene.unity
Network MIDI (WSNet2)	Assets/MIDI/Samples/Integrations/Networking/Scenes/MidiWsnet2NetworkSampleScene.unity
Scriptable Audio integration	Assets/MIDI/Samples/Integrations/ScriptableAudio/Scenes/ScriptableAudioMetronomeSampleScene.unity
Scriptable Audio SMF sequence	Assets/MIDI/Samples/Integrations/ScriptableAudio/Scenes/ScriptableAudioSequenceSampleScene.unity
Scriptable Audio UMP sequence	Assets/MIDI/Samples/Integrations/ScriptableAudio/Scenes/ScriptableAudioUmpSequenceSampleScene.unity
Chunity bridge	Assets/MIDI/Samples/Integrations/Chunity/Scenes/ChunityBridgeSampleScene.unity
Chunity workflow	Assets/MIDI/Samples/Integrations/Chunity/Scenes/ChunityWorkflowsSampleScene.unity
Chunity presets	Assets/MIDI/Samples/Integrations/Chunity/Scenes/ChunityPresetsSampleScene.unity

Chunity mic FX	Assets/MIDI/Samples/Integrations/Chunity/Scenes/ChunityMicFxSampleScene.unity
Chunity Generator Workflow	Assets/MIDI/Samples/Integrations/Chunity/Scenes/ChunityGeneratorWorkflowSampleScene.unity

Each scene supports IMGUI testing with virtual MIDI input. For a detailed procedure, see [Samples](#).

The Animator `MidiAnimatorSample.controller` is bundled in `Samples/Integrations/Resources/`. To regenerate it, run `Window > MIDI > Samples > Generate Integration Sample Assets`.

VST3 Plugin Host (separate package)

VST3 instrument/effect hosting is **not** part of this MIDI plugin and is **not** shipped in MIDI releases (`VstHostNative.dll` , VST3 SDK, and VST host C# are excluded).

Use the separate package:

Item	Value
Display name	Unity Plugin Host for VST3
UPM / Git URL	<code>https://github.com/kshoji/Unity-VST3-Bridge.git</code>
Package id	<code>jp.kshoji.unity.vst3nativehost</code>

Setup, scan paths, and optional MIDI wiring live in that repository (`Documentation~/usage.md` , `midi-integration.md`). This MIDI package only exposes public MIDI events for subscribers; do not add VST implementation under `Assets/MIDI` . Desktop Player MCP for VST lives in the Bridge package; **Android device MCP is not a VST3 target**.

Optional MIDI helpers in the VST package (requires `FEATURE_MIDI_PLUGIN`):

Feature	Components
CC / pitch bend → VST parameters	<code>VstMidiParameterMapping</code> , <code>VstHostMidiParameterMapper</code>
SMF → VST instrument	<code>VstHostSmfLink</code> + <code>SmfPlayer.outputDeviceId</code> → virtual device → <code>VstHostMidiAdapter</code> (optional channel routes)
Presets / A/B	<code>VstPresetAsset</code> , <code>VstPresetBrowser</code> , Window → VST3 Host → Preset Browser

Additional VST package integrations (Timeline / Input System packages enable optional assemblies automatically):

Feature	Components
Timeline parameter automation	<code>VstParameterTrack</code> / <code>VstParameterClip</code> , <code>VstProgramChangeMarker</code> (<code>FEATURE_USE_TIMELINE</code>)
Animator ↔ VST	<code>VstAnimatorMapping</code> , <code>VstAnimatorDriver</code>
Input System → VST	<code>InputSystemToVstBridge</code> (<code>FEATURE_INPUT_SYSTEM</code>) or MIDI <code>InputSystemToMidiBridge</code> → Adapter
Editor tools	Plugin Browser (category + vendor/tag), Activity Monitor, Virtual Controller, Project Settings → VST3 Host
Plugin chain	<code>VstPluginChain</code> , <code>VstHostChannelRouteSync</code> , <code>VstHostMidiFilterLink</code> , <code>VstHostEventSink</code>
Scriptable Audio (Unity 6.3+)	<code>VstHostGenerator</code> , <code>VstHostDspMidiOutBridge</code> → scheduler <code>extraTimedMidiOutput</code>
Visual Scripting	VST3 Host units / events (<code>FEATURE_USE_VISUALSCRIPTING</code>)
Network MIDI → VST	<code>VstHostNetworkMidiLink</code> (<code>FEATURE_MIDI_NETWORK</code>)
Chunity ↔ VST	<code>VstHostChuckEventMidiLink</code> , <code>VstHostChuckEffectBridge</code> (<code>FEATURE_CHUNITY</code>)

Details: VST package `Documentation~/timeline.md`, `animator-input.md`, `editor-tools.md`, `plugin-chain.md`, `scriptable-audio.md`, `visual-scripting.md`, `network-midi.md`, `chunity.md`.

Related documents

- [Gameplay Components](#) — `SmfPlayer` / `MidiRecorder` / `MidiInputRouter` / `MidiClockSync`
- [SMF Tools](#) — `MidiSequenceAsset` / `UmpSequenceAsset` / `TempoMapExtractor`
- [Utilities](#) — `MidiNoteUtility` / `MidiMessageBuilder`
- [Build Post-processing](#) — Enabling optional integration symbols
- [Samples](#) — Procedures for integration sample scenes
- [Embedded Third-Party Modules](#) — Vendored modules (VST host is a separate package, not bundled here)